

Métodos iterativos estacionarios para sistemas lineales

Clase 7 · Capítulo 4

Análisis Numérico (LMA) · Métodos Numéricos II (LCD)

8 de septiembre de 2026

Facultad de Ingeniería Química · UNL

Clase 7

1. Convergencia de Jacobi (A e.d.d.).
2. Convergencia de Gauss-Seidel (A e.d.d.).
3. Convergencia de SOR (A sdp).
4. Richardson óptimo y $\kappa_2(A)$.

4.2 Métodos iterativos estacionarios

Esquema general: métodos estacionarios

Todos los métodos de esta sección tienen la forma

$$x_{k+1} = M x_k + c, \quad k = 0, 1, 2, \dots$$

con M (matriz de iteración) y c fijos.

Si x es la solución de $Ax = b$, entonces $x = Mx + c$, y el **error** $e_k = x_k - x$ satisface

$$e_{k+1} = M e_k = M^{k+1} e_0.$$

Teorema

La iteración $x_{k+1} = Mx_k + c$ converge para todo x_0 **si y solo si** $\rho(M) < 1$, donde $\rho(M) = \max\{|\lambda| : \lambda \text{ autovalor de } M\}$ es el radio espectral de M .

Método de Jacobi

De $Ax = b$ se despeja la i -ésima componente:

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j \right).$$

Iteración de Jacobi

$$x_{k+1}^i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_k^j \right), \quad i = 1, \dots, n.$$

En forma matricial, con $A = L + D + U$:

$$x_{k+1} = \underbrace{-D^{-1}(L + U)}_{M_J} x_k + D^{-1}b.$$

$$(M_J)_{ij} = -a_{ij}/a_{ii} \text{ para } i \neq j; \quad (M_J)_{ii} = 0.$$

Convergencia de Jacobi: matrices e.d.d.

Recordemos la iteración, con $A = L + D + U$:

$$x_{k+1} = D^{-1}[b - (L + U)x_k] = \underbrace{-D^{-1}(L + U)}_{M_J} x_k + \underbrace{D^{-1}b}_c.$$

Definición (Matriz estrictamente diagonalmente dominante)

$A \in \mathbb{R}^{n \times n}$ es **e.d.d.** si $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ para todo i .

Convergencia de Jacobi: matrices e.d.d.

Recordemos la iteración, con $A = L + D + U$:

$$x_{k+1} = D^{-1}[b - (L + U)x_k] = \underbrace{-D^{-1}(L + U)}_{M_J} x_k + \underbrace{D^{-1}b}_c.$$

Definición (Matriz estrictamente diagonalmente dominante)

$A \in \mathbb{R}^{n \times n}$ es **e.d.d.** si $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ para todo i .

Teorema (Convergencia de Jacobi)

Si A es e.d.d., Jacobi converge para todo x_0 y

$$\|x_{k+1} - x\|_{\infty} \leq \mu_J \|x_k - x\|_{\infty},$$

$$\text{con } \mu_J = \max_i \frac{\sum_{j \neq i} |a_{ij}|}{|a_{ii}|} < 1.$$

Convergencia de Jacobi: matrices e.d.d.

Recordemos la iteración, con $A = L + D + U$:

$$x_{k+1} = D^{-1}[b - (L + U)x_k] = \underbrace{-D^{-1}(L + U)}_{M_J} x_k + \underbrace{D^{-1}b}_c.$$

Definición (Matriz estrictamente diagonalmente dominante)

$A \in \mathbb{R}^{n \times n}$ es **e.d.d.** si $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ para todo i .

Teorema (Convergencia de Jacobi)

Si A es e.d.d., Jacobi converge para todo x_0 y

$$\|x_{k+1} - x\|_{\infty} \leq \mu_J \|x_k - x\|_{\infty},$$

$$\text{con } \mu_J = \max_i \frac{\sum_{j \neq i} |a_{ij}|}{|a_{ii}|} < 1.$$

Idea: $\|M_J\|_{\infty} = \mu_J < 1$ por ser A e.d.d.

Método de Gauss-Seidel

Idea: al calcular x_{k+1}^i , **usar ya** los valores $x_{k+1}^1, \dots, x_{k+1}^{i-1}$ (disponibles si se procesa en orden).

Método de Gauss-Seidel

Idea: al calcular x_{k+1}^i , **usar ya** los valores $x_{k+1}^1, \dots, x_{k+1}^{i-1}$ (disponibles si se procesa en orden).

Iteración de Gauss-Seidel

$$x_{k+1}^i = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_{k+1}^j - \sum_{j > i} a_{ij} x_k^j \right).$$

Método de Gauss-Seidel

Idea: al calcular x_{k+1}^i , **usar ya** los valores $x_{k+1}^1, \dots, x_{k+1}^{i-1}$ (disponibles si se procesa en orden).

Iteración de Gauss-Seidel

$$x_{k+1}^i = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_{k+1}^j - \sum_{j > i} a_{ij} x_k^j \right).$$

En forma matricial:

$$x_{k+1} = \underbrace{-(L + D)^{-1}U}_{M_{GS}} x_k + (L + D)^{-1}b.$$

Método de Gauss-Seidel

Idea: al calcular x_{k+1}^i , **usar ya** los valores $x_{k+1}^1, \dots, x_{k+1}^{i-1}$ (disponibles si se procesa en orden).

Iteración de Gauss-Seidel

$$x_{k+1}^i = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_{k+1}^j - \sum_{j > i} a_{ij} x_k^j \right).$$

En forma matricial:

$$x_{k+1} = \underbrace{-(L + D)^{-1}U}_{M_{GS}} x_k + (L + D)^{-1}b.$$

- No requiere almacenar dos copias del vector: actualización *in-place*.

Convergencia de Gauss-Seidel: matrices e.d.d.

Recordemos la iteración, con $A = L + D + U$:

$$x_{k+1} = (L + D)^{-1} [b - Ux_k] = \underbrace{-(L + D)^{-1}U}_{M_{GS}} x_k + \underbrace{(L + D)^{-1}b}_c.$$

Teorema

Si A es e.d.d., Gauss-Seidel converge, con $\|x_{k+1} - x\|_\infty \leq \mu_{GS} \|x_k - x\|_\infty$,

El factor de reducción es $\mu_{GS} := \max_i \frac{\beta_i}{1 - \alpha_i} \leq \mu_J$,

$$\text{con } \alpha_i = \sum_{j < i} \frac{|a_{ij}|}{|a_{ii}|} \text{ y } \beta_i = \sum_{j > i} \frac{|a_{ij}|}{|a_{ii}|}.$$

Convergencia de Gauss-Seidel: matrices e.d.d.

Recordemos la iteración, con $A = L + D + U$:

$$x_{k+1} = (L + D)^{-1} [b - Ux_k] = \underbrace{-(L + D)^{-1}U}_{M_{GS}} x_k + \underbrace{(L + D)^{-1}b}_c.$$

Teorema

Si A es e.d.d., Gauss-Seidel converge, con $\|x_{k+1} - x\|_\infty \leq \mu_{GS} \|x_k - x\|_\infty$,

El factor de reducción es $\mu_{GS} := \max_i \frac{\beta_i}{1 - \alpha_i} \leq \mu_J$,

$$\text{con } \alpha_i = \sum_{j < i} \frac{|a_{ij}|}{|a_{ii}|} \text{ y } \beta_i = \sum_{j > i} \frac{|a_{ij}|}{|a_{ii}|}.$$

Conclusión: matrices e.d.d., Gauss-Seidel converge al menos tan rápido como Jacobi.

Convergencia de Gauss-Seidel: matrices e.d.d.

Recordemos la iteración, con $A = L + D + U$:

$$x_{k+1} = (L + D)^{-1} [b - Ux_k] = \underbrace{-(L + D)^{-1}U}_{M_{GS}} x_k + \underbrace{(L + D)^{-1}b}_c.$$

Teorema

Si A es e.d.d., Gauss-Seidel converge, con $\|x_{k+1} - x\|_\infty \leq \mu_{GS} \|x_k - x\|_\infty$,

El factor de reducción es $\mu_{GS} := \max_i \frac{\beta_i}{1 - \alpha_i} \leq \mu_J$,

$$\text{con } \alpha_i = \sum_{j < i} \frac{|a_{ij}|}{|a_{ii}|} \text{ y } \beta_i = \sum_{j > i} \frac{|a_{ij}|}{|a_{ii}|}.$$

Conclusión: matrices e.d.d., Gauss-Seidel converge al menos tan rápido como Jacobi.

Observación: e.d.d. es suficiente pero no necesaria.

Método SOR (sobrerrelajación sucesiva)

Dado $\omega \in \mathbb{R} \setminus \{0\}$, se parte de la descomposición $A = \left(\frac{1}{\omega}D + L\right) + \left(\frac{\omega-1}{\omega}D + U\right)$.

Iteración SOR

$$x_{k+1} = \underbrace{-\left(\frac{1}{\omega}D + L\right)^{-1}\left(\frac{\omega-1}{\omega}D + U\right)}_{M_{SOR}^{\omega}} x_k + \left(\frac{1}{\omega}D + L\right)^{-1}b.$$

- $\omega = 1$: Gauss-Seidel.
- $\omega \in (1, 2)$: *sobre*-relajación (acelera la convergencia en muchos casos).
- $\omega \in (0, 1)$: *sub*-relajación (puede estabilizar métodos divergentes).

Necesidad de $\omega \in (0, 2)$

Si $\omega \notin (0, 2)$, entonces $\rho(M_{SOR}^{\omega}) \geq 1$ para **toda** A invertible. No hay convergencia posible.

Convergencia de SOR: matrices sd

Recordemos la iteración, con $A = A_1 + A_2$, $A_1 = \frac{1}{\omega}D + L$, $A_2 = \frac{\omega-1}{\omega}D + U$:

$$x_{k+1} = A_1^{-1}[b - A_2 x_k] = \underbrace{-A_1^{-1}A_2}_{M_{SOR}^\omega} x_k + \underbrace{A_1^{-1}b}_c.$$

Teorema (Convergencia de SOR)

Si A es *sd* y $\omega \in (0, 2)$, entonces el método SOR converge para todo $x_0 \in \mathbb{R}^n$.

Convergencia de SOR: matrices sdp

Recordemos la iteración, con $A = A_1 + A_2$, $A_1 = \frac{1}{\omega}D + L$, $A_2 = \frac{\omega-1}{\omega}D + U$:

$$x_{k+1} = A_1^{-1}[b - A_2 x_k] = \underbrace{-A_1^{-1}A_2}_{M_{SOR}^{\omega}} x_k + \underbrace{A_1^{-1}b}_c.$$

Teorema (Convergencia de SOR)

Si A es *sdp* y $\omega \in (0, 2)$, entonces el método SOR converge para todo $x_0 \in \mathbb{R}^n$.

- La demostración se reduce a verificar que $\|M_{SOR}^{\omega}\|_A < 1$.

Convergencia de SOR: matrices sd

Recordemos la iteración, con $A = A_1 + A_2$, $A_1 = \frac{1}{\omega}D + L$, $A_2 = \frac{\omega-1}{\omega}D + U$:

$$x_{k+1} = A_1^{-1}[b - A_2 x_k] = \underbrace{-A_1^{-1}A_2}_{M_{SOR}^\omega} x_k + \underbrace{A_1^{-1}b}_c.$$

Teorema (Convergencia de SOR)

Si A es *sd* y $\omega \in (0, 2)$, entonces el método SOR converge para todo $x_0 \in \mathbb{R}^n$.

- La demostración se reduce a verificar que $\|M_{SOR}^\omega\|_A < 1$.
- El ω que minimiza $\rho(M_{SOR}^\omega)$ depende de los autovalores de A y hallarlo puede ser costoso; en la práctica se exploran valores $\omega \in (1, 2)$.

Convergencia de SOR: matrices sdp

Recordemos la iteración, con $A = A_1 + A_2$, $A_1 = \frac{1}{\omega}D + L$, $A_2 = \frac{\omega-1}{\omega}D + U$:

$$x_{k+1} = A_1^{-1}[b - A_2 x_k] = \underbrace{-A_1^{-1}A_2}_{M_{SOR}^\omega} x_k + \underbrace{A_1^{-1}b}_c.$$

Teorema (Convergencia de SOR)

Si A es *sdp* y $\omega \in (0, 2)$, entonces el método SOR converge para todo $x_0 \in \mathbb{R}^n$.

- La demostración se reduce a verificar que $\|M_{SOR}^\omega\|_A < 1$.
- El ω que minimiza $\rho(M_{SOR}^\omega)$ depende de los autovalores de A y hallarlo puede ser costoso; en la práctica se exploran valores $\omega \in (1, 2)$.

¿Por qué $\omega \in (0, 2)$ es necesario?

$\det(M_{SOR}^\omega) = (1 - \omega)^n$, así que $|\omega - 1| \geq 1 \Rightarrow \rho(M_{SOR}^\omega) \geq 1$.

$$Ax = b \iff x = x - \alpha(Ax - b) \iff x = \underbrace{(I - \alpha A)}_M x + \alpha b$$

Iteración de Richardson ($\alpha \neq 0$ dado)

$$x_{k+1} = (I - \alpha A) x_k + \alpha b.$$

- Matriz de iteración: $M_R = I - \alpha A$.
- Solo requiere el producto $x \mapsto Ax$ (no las entradas de A).
- ¿Cómo elegir α ?

$$x_{k+1} = x_k - \alpha(Ax_k - b) = \underbrace{(I - \alpha A)}_{M_R} x_k + \underbrace{\alpha b}_c.$$

Sea A sdp con autovalores $0 < \lambda_{\min} \leq \dots \leq \lambda_{\max}$. Los autovalores de $M_R = I - \alpha A$ son $1 - \alpha\lambda$, con $\lambda \in \sigma(A)$.

$$x_{k+1} = x_k - \alpha(Ax_k - b) = \underbrace{(I - \alpha A)}_{M_R} x_k + \underbrace{\alpha b}_c.$$

Sea A sdp con autovalores $0 < \lambda_{\min} \leq \dots \leq \lambda_{\max}$. Los autovalores de $M_R = I - \alpha A$ son $1 - \alpha\lambda$, con $\lambda \in \sigma(A)$.

$$\rho(I - \alpha A) = \max\{|1 - \alpha\lambda_{\max}|, |1 - \alpha\lambda_{\min}|\}.$$

Convergencia de Richardson: elección de α

$$x_{k+1} = x_k - \alpha(Ax_k - b) = \underbrace{(I - \alpha A)}_{M_R} x_k + \underbrace{\alpha b}_c.$$

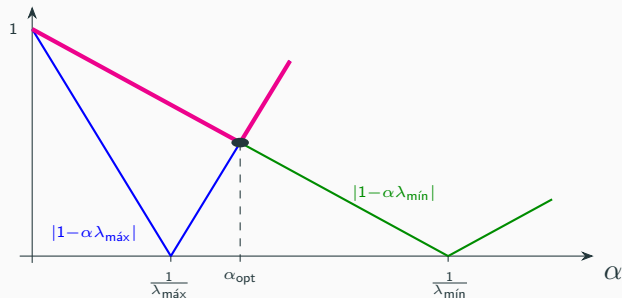
Sea A sdp con autovalores $0 < \lambda_{\min} \leq \dots \leq \lambda_{\max}$. Los autovalores de $M_R = I - \alpha A$ son $1 - \alpha\lambda$, con $\lambda \in \sigma(A)$.

$$\rho(I - \alpha A) = \max\{|1 - \alpha\lambda_{\max}|, |1 - \alpha\lambda_{\min}|\}.$$

α **óptimo**: minimizar $\rho(I - \alpha A)$.

El α óptimo, gráficamente

$$\rho(I - \alpha A) = \max\{|1 - \alpha\lambda_{\text{máx}}|, |1 - \alpha\lambda_{\text{mín}}|\}$$



$$\alpha\lambda_{\text{máx}} - 1 = 1 - \alpha\lambda_{\text{mín}} \implies \alpha_{\text{opt}} = \frac{2}{\lambda_{\text{mín}} + \lambda_{\text{máx}}}.$$

Richardson óptimo y número de condición

$$\text{Con } \alpha_{\text{opt}} = \frac{2}{\lambda_{\text{mín}} + \lambda_{\text{máx}}}: \quad \rho(I - \alpha_{\text{opt}}A) = \frac{\lambda_{\text{máx}} - \lambda_{\text{mín}}}{\lambda_{\text{máx}} + \lambda_{\text{mín}}} = \frac{\kappa_2(A) - 1}{\kappa_2(A) + 1}.$$

Richardson óptimo y número de condición

$$\text{Con } \alpha_{\text{opt}} = \frac{2}{\lambda_{\text{mín}} + \lambda_{\text{máx}}}: \quad \rho(I - \alpha_{\text{opt}}A) = \frac{\lambda_{\text{máx}} - \lambda_{\text{mín}}}{\lambda_{\text{máx}} + \lambda_{\text{mín}}} = \frac{\kappa_2(A) - 1}{\kappa_2(A) + 1}.$$

Teorema

Con $\alpha = \alpha_{\text{opt}}$ y A sdp, para todo x_0 :

$$\|x_{k+1} - x\|_{\bullet} \leq \frac{\kappa_2(A) - 1}{\kappa_2(A) + 1} \|x_k - x\|_{\bullet}, \quad \|\cdot\|_{\bullet} = \|\cdot\|_2 \text{ o } \|\cdot\|_A.$$

Richardson óptimo y número de condición

$$\text{Con } \alpha_{\text{opt}} = \frac{2}{\lambda_{\text{mín}} + \lambda_{\text{máx}}}: \quad \rho(I - \alpha_{\text{opt}}A) = \frac{\lambda_{\text{máx}} - \lambda_{\text{mín}}}{\lambda_{\text{máx}} + \lambda_{\text{mín}}} = \frac{\kappa_2(A) - 1}{\kappa_2(A) + 1}.$$

Teorema

Con $\alpha = \alpha_{\text{opt}}$ y A sdp, para todo x_0 :

$$\|x_{k+1} - x\|_{\bullet} \leq \frac{\kappa_2(A) - 1}{\kappa_2(A) + 1} \|x_k - x\|_{\bullet}, \quad \|\cdot\|_{\bullet} = \|\cdot\|_2 \text{ o } \|\cdot\|_A.$$

Impacto del número de condición

$\kappa_2(A)$	10	100	1000
Factor de reducción	0.82	0.98	0.998

Matrices **mal condicionadas** hacen muy lenta la convergencia.

Resumen: condiciones de convergencia

Método	Condición suficiente	Cond. nec. y suf.
Richardson	A sdp, $0 < \alpha < 2/\lambda_{\text{máx}}$	$\rho(I - \alpha A) < 1$
Jacobi	A e.d.d.	$\rho(M_J) < 1$
Gauss-Seidel	A e.d.d. (o A sdp)	$\rho(M_{GS}) < 1$
SOR	A sdp, $\omega \in (0, 2)$	$\rho(M_{SOR}^{\omega}) < 1$

Resumen: condiciones de convergencia

Método	Condición suficiente	Cond. nec. y suf.
Richardson	A sdp, $0 < \alpha < 2/\lambda_{\max}$	$\rho(I - \alpha A) < 1$
Jacobi	A e.d.d.	$\rho(M_J) < 1$
Gauss-Seidel	A e.d.d. (o A sdp)	$\rho(M_{GS}) < 1$
SOR	A sdp, $\omega \in (0, 2)$	$\rho(M_{SOR}^{\omega}) < 1$

- La condición necesaria y suficiente **siempre** es $\rho(M) < 1$.
- Las condiciones suficientes son las más útiles en la práctica.
- Para e.d.d.: $\mu_{GS} \leq \mu_J < 1$ (Gauss-Seidel no es peor que Jacobi).

Jacobi

```
function x = jacobi(A, b, x0, tol, maxit)
    D = diag(diag(A));
    LU = A - D;
    x = x0;
    for k = 1:maxit
        x_new = D \ (b - LU*x);
        if norm(x_new - x) < tol, break; end
        x = x_new;
    end
end
```

Gauss-Seidel / SOR

```
function x = sor(A, b, x0, omega, tol, maxit)
    n = length(b);
    x = x0;
    for k = 1:maxit
        x_old = x;
        for i = 1:n
            s = b(i) - A(i,:)*x + A(i,i)*x(i);
            x(i) = (1-omega)*x(i) + omega*s/A(i,i);
        end
        if norm(x-x_old) < tol, break; end
    end
end
```

$\omega = 1$: Gauss-Seidel. Funciones: norm, cond.

Para recordar

1. Los métodos iterativos estacionarios tienen la forma $x_{k+1} = Mx_k + c$.
2. **Convergen** $\iff \rho(M) < 1$; cuanto menor $\rho(M)$, más rápido.
3. **Jacobi** y **Gauss-Seidel**: convergen si A es e.d.d.
4. **SOR**: converge si A sdp y $\omega \in (0, 2)$; el ω óptimo reduce ρ .
5. **Richardson**: converge si A sdp y $0 < \alpha < 2/\lambda_{\text{máx}}$. Con α_{opt} , el factor de reducción es $(\kappa_2 - 1)/(\kappa_2 + 1)$.
6. El **número de condición** gobierna la velocidad: matrices mal condicionadas requieren muchas iteraciones.